

Scripting, Automation & Web Security

Week 7: Web Application Foundations & SQL Injection Mastery

Submitted By: Muhammad Usama Bilal

Roll Number : CSI-B1-460

INSTRUCTOR: MUHAMMAD SAAD

Date: April 13 ,2026

Contents

Abstract 3

1. Introduction..... 3

2. Lab Environment..... 4

3. Vulnerable Application Configuration..... 4

4. Intercepting and Analyzing Traffic..... 5

5. SQL Injection Exploitation 6

.6 Automated Data Extraction..... 8

7. Findings & Recommendations 9

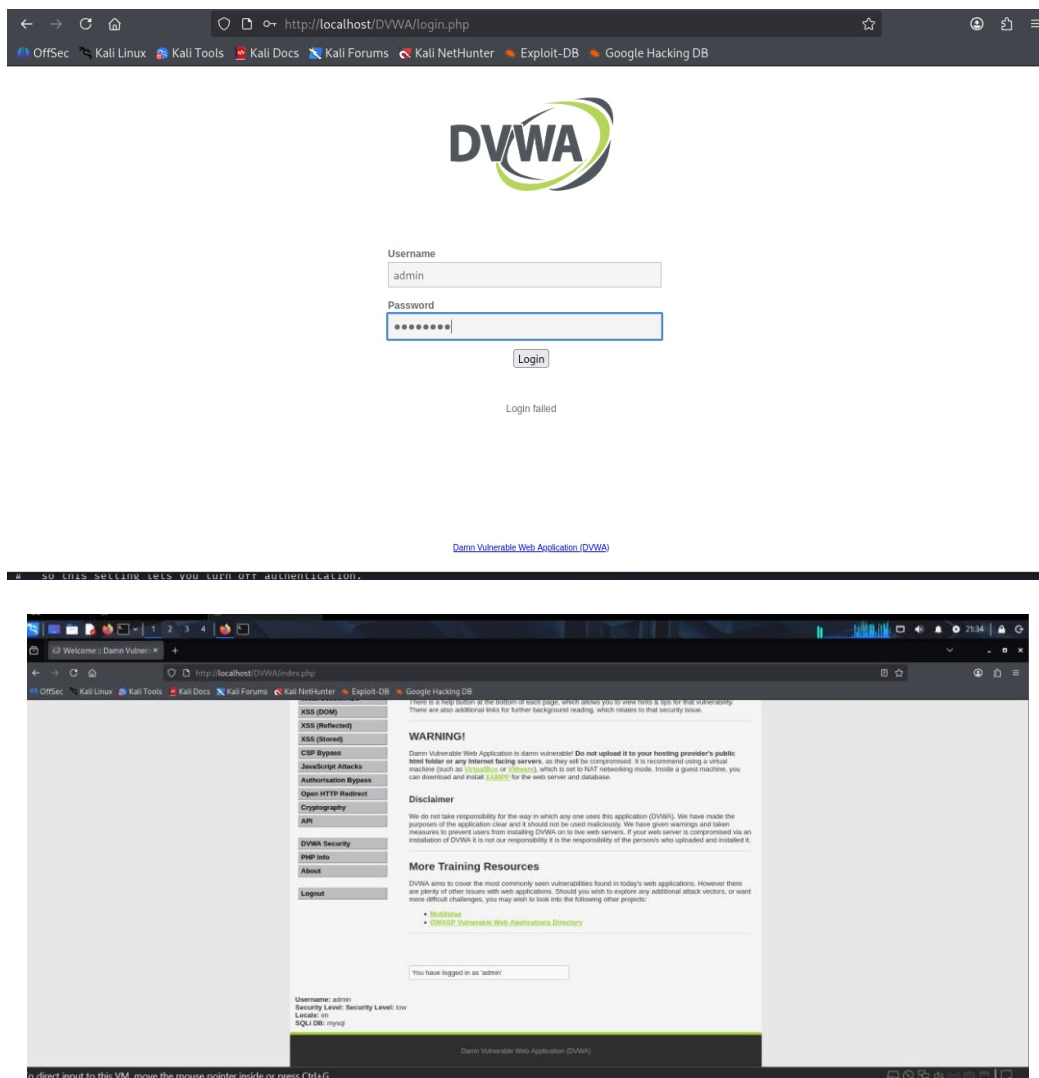
8. Conclusion 9

Abstract

This report details the exploration, identification, and exploitation of SQL Injection (SQLi) vulnerabilities within a controlled web environment. The study transitions from manual traffic inspection to automated database extraction, emphasizing the critical need for robust input validation and secure coding practices in modern web architectures.

1. Introduction

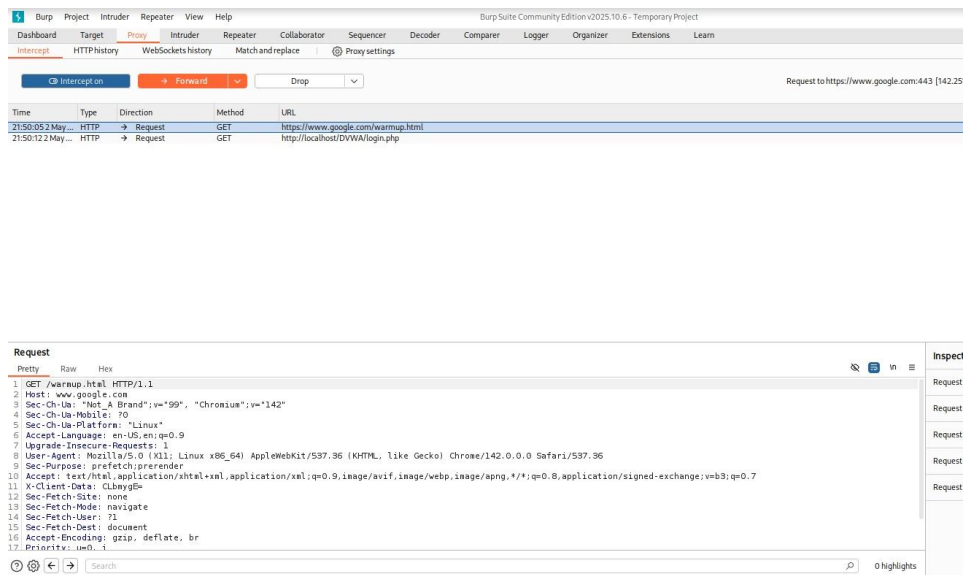
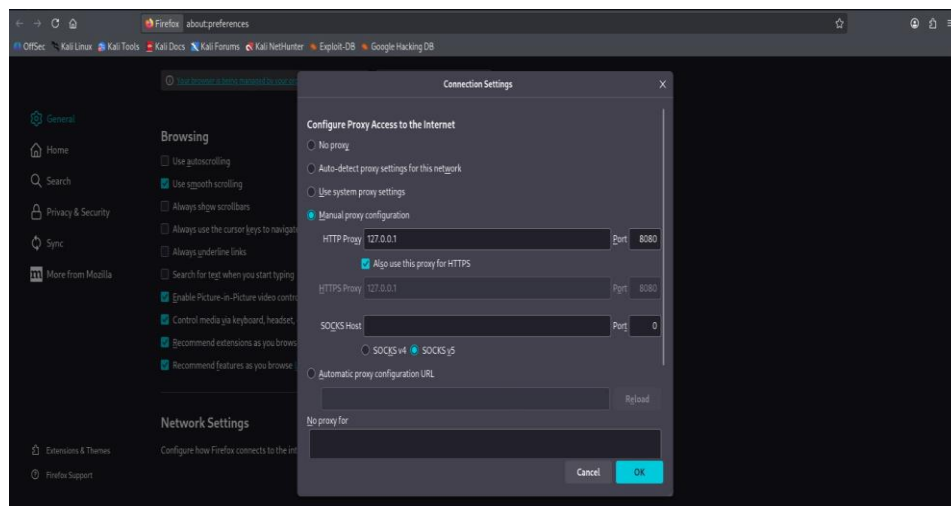
Web applications are often the primary target for attackers seeking unauthorized data access. This lab focuses on understanding the mechanics of SQL Injection—a vulnerability where malicious SQL statements are inserted into entry fields for execution. By utilizing industry-standard tools like Burp Suite and SQLMap, we simulate real-world attack vectors to assess and mitigate database-level risks.



2. Lab Environment

To conduct these tests, a local security laboratory was established using the following components:

- **Target Application:** DVWA (Damn Vulnerable Web Application).
- **Server Stack:** XAMPP (Apache & MySQL/MariaDB).
- **Proxy & Analysis:** Burp Suite Community Edition.
- **Automation:** SQLMap.
- **Operating System:** Kali Linux.



3. Vulnerable Application Configuration

The initial phase involved deploying the DVWA environment.

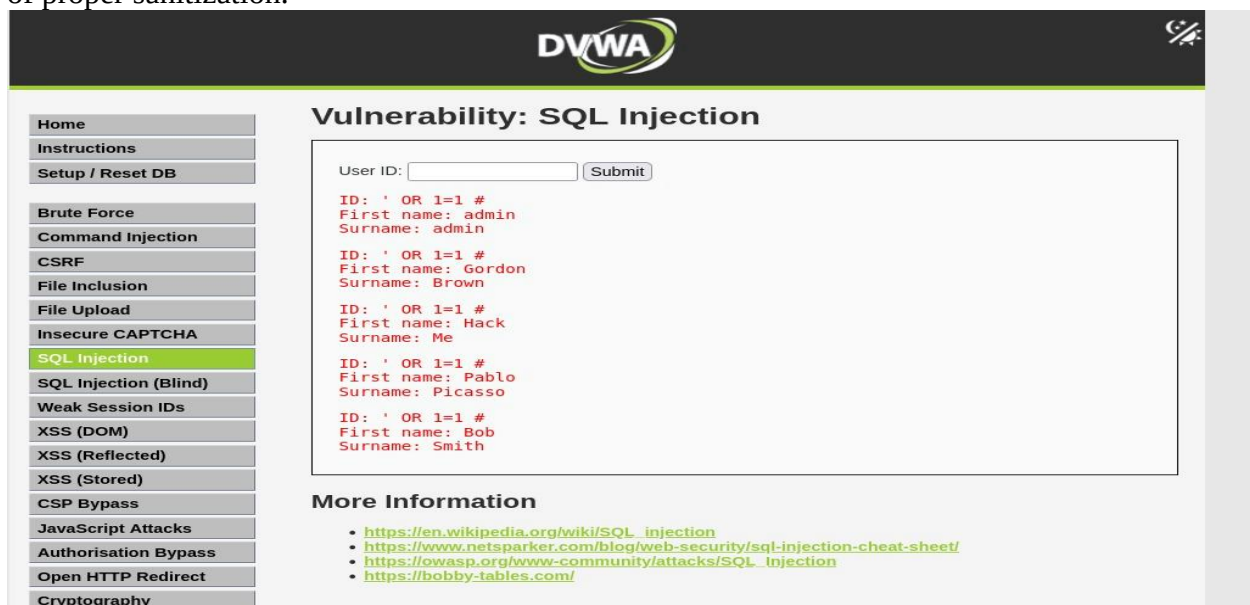
- **Database Setup:** MariaDB was initialized, and a dedicated user (`dvwa_user`) was created with full privileges over the `dvwa` database.
- **Application Tuning:** The `config.inc.php` file was modified to reflect the database credentials.

- **Security Level:** The application was set to "Low" security to allow for fundamental SQLi testing.

4. Intercepting and Analyzing Traffic

Using Burp Suite as a manual intercepting proxy, HTTP traffic between the browser and the web server was analyzed.

- **Proxy Configuration:** The browser was configured to route traffic through 127.0.0.1:8080.
- **Request Inspection:** Captured GET and POST requests were examined to identify user-controllable parameters, specifically the `id` and `login` fields.
- **Observation:** Manipulation of these parameters directly influenced server-side queries, indicating a lack of proper sanitization. –



The screenshot displays the DVWA (Damn Vulnerable Web Application) interface. The top header features the DVWA logo and a security level indicator set to 'Low'. On the left, a sidebar menu lists various attack types, with 'SQL Injection' currently selected and highlighted in green. The main content area is titled 'Vulnerability: SQL Injection' and contains a login form with a 'User ID:' input field and a 'Submit' button. Below the form, the application displays the results of four successful SQL injection attacks, each showing the retrieved user details in red text:

- ID: ' OR 1=1 #
First name: admin
Surname: admin
- ID: ' OR 1=1 #
First name: Gordon
Surname: Brown
- ID: ' OR 1=1 #
First name: Hack
Surname: Me
- ID: ' OR 1=1 #
First name: Pablo
Surname: Picasso
- ID: ' OR 1=1 #
First name: Bob
Surname: Smith

Below the results, a 'More Information' section provides links to external resources for further learning about SQL injection:

- https://en.wikipedia.org/wiki/SQL_injection
- <https://www.netsparker.com/blog/web-security/sql-injection-cheat-sheet/>
- https://owasp.org/www-community/attacks/SQL_injection
- <https://bobby-tables.com/>

DVWA

Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
XSS (Stored)
CSP Bypass
JavaScript Attacks
Authorisation Bypass
Open HTTP Redirect
Cryptography

Vulnerability: SQL Injection

User ID:

ID: ' UNION SELECT user(), version() #
First name: dvwa user@localhost
Surname: 11.8.3-MariaDB-1+b1 from Debian

More Information

- https://en.wikipedia.org/wiki/SQL_injection
- <https://www.netsparker.com/blog/web-security/sql-injection-cheat-sheet/>
- https://owasp.org/www-community/attacks/SQL_injection
- <https://bobby-tables.com/>

5. SQL Injection Exploitation

5.1 Manual Injection (Error & Union Based)

Initial testing used simple logic bypasses and data extraction techniques:

- **Auth Bypass:** Using ' OR 1=1 # as a payload successfully returned all user records from the database, bypassing standard authentication logic.
- **Database Enumeration:** The UNION SELECT statement was used to identify the database name (dvwa) and the current user (dvwa_user@localhost).
- **Structure Discovery:** Queries were crafted to pull table names (e.g., users, guestbook) and column names (e.g., user, password, avatar) from the information_schema.

DVWA

Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
XSS (Stored)
CSP Bypass
JavaScript Attacks
Authorisation Bypass
Open HTTP Redirect
Cryptography

Vulnerability: SQL Injection

User ID:

ID: ' UNION SELECT null, database() #
First name: dvwa
Surname: dvwa

More Information

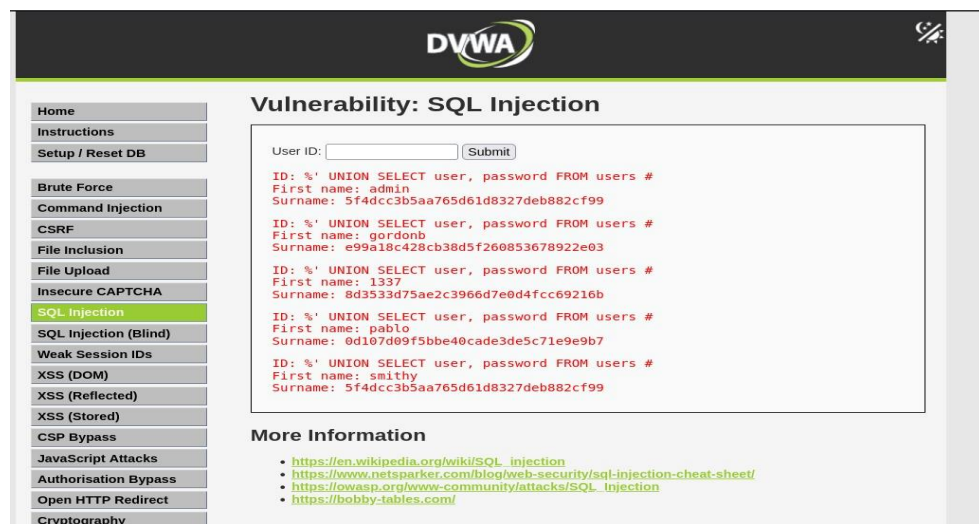
- https://en.wikipedia.org/wiki/SQL_injection
- <https://www.netsparker.com/blog/web-security/sql-injection-cheat-sheet/>
- https://owasp.org/www-community/attacks/SQL_injection
- <https://bobby-tables.com/>

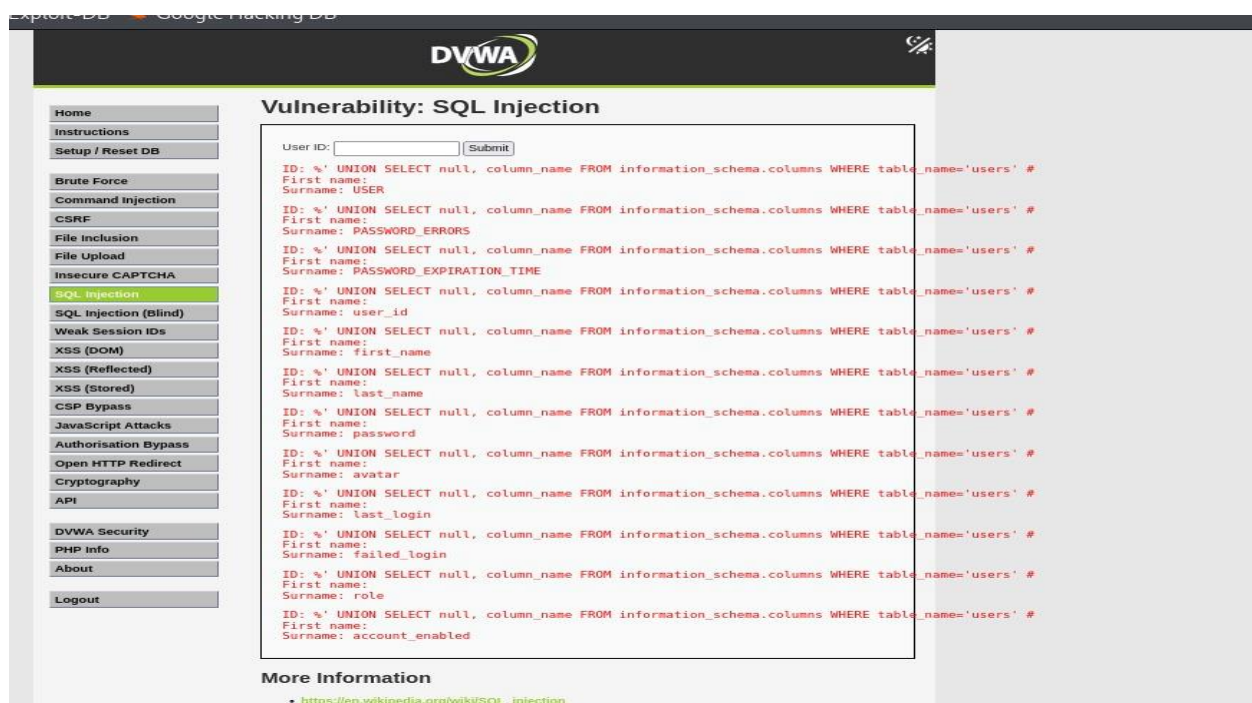


5.2 Advanced Injection (Blind & Time-Based)

In scenarios where no data is directly reflected on the page, more subtle techniques were employed:

- **Boolean-Based:** Inferred data by observing changes in the page's response when submitting true vs. false conditions (e.g., `AND 1=1` vs. `AND 1=2`).
- **Time-Based:** Confirmed the vulnerability by injecting `SLEEP(5)` commands, where a delayed server response confirmed that the query was being executed.

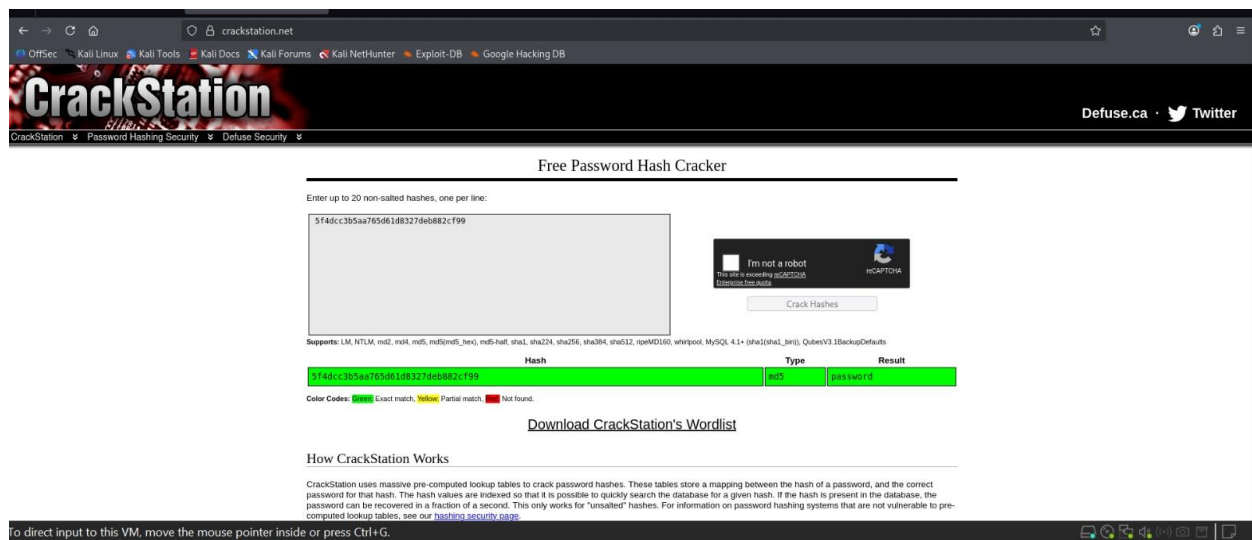


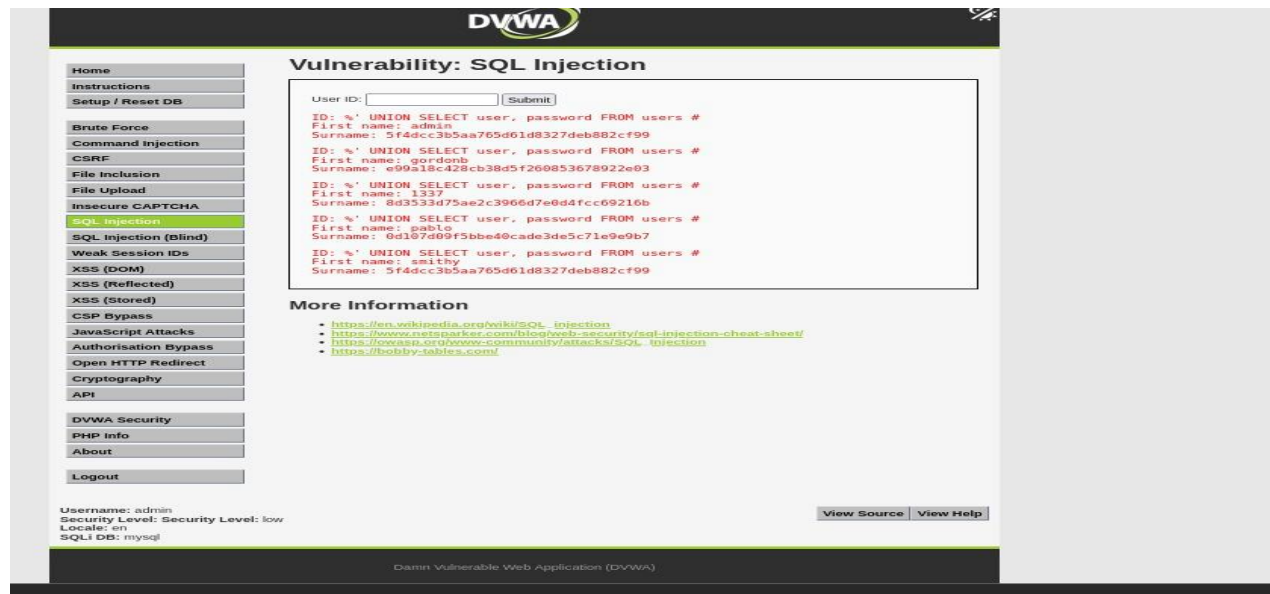


6 Automated Data Extraction

SQLMap was utilized to demonstrate the speed and efficiency of automated exploitation.

- **Command Execution:** By providing the intercepted session cookie and the target URL, SQLMap automatically identified the injection points.
- **Data Dumping:** The tool successfully dumped the entire `users` table, including administrative usernames and hashed passwords





7. Findings & Recommendations

Key Findings

- **Total Exposure:** Sensitive user credentials and database structures were fully compromised.
- **Input Failure:** The application accepted raw SQL commands through standard input fields.
- **Tool Efficiency:** While manual testing provides understanding, automated tools like SQLMap significantly accelerate the data exfiltration process.

Security Recommendations

- **Parameterized Queries:** Use Prepared Statements to ensure the database treats user input as data, not executable code.
- **Input Sanitization:** Implement strict allow-lists for user input.
- **Error Management:** Disable verbose database error messages that reveal structural information to users.
- **WAF Deployment:** Utilize a Web Application Firewall to block common SQLi patterns.

8. Conclusion

The lab successfully showcased the lifecycle of a SQL Injection attack, from initial environment setup to full database compromise. The results highlight that without rigorous security controls like parameterized queries and input validation, web applications remains highly susceptible to unauthorized data access and structural exposure.